

DEEP MATERIAL NETWORK FOR MULTISCALE TOPOLOGY LEARNING

ME 618: Micromechanics of Material

Kumar Shailendra 230103125

Devanandika P 230103033

PROBLEM OVERVIEW AND SOLUTION OVERVIEW

Problem

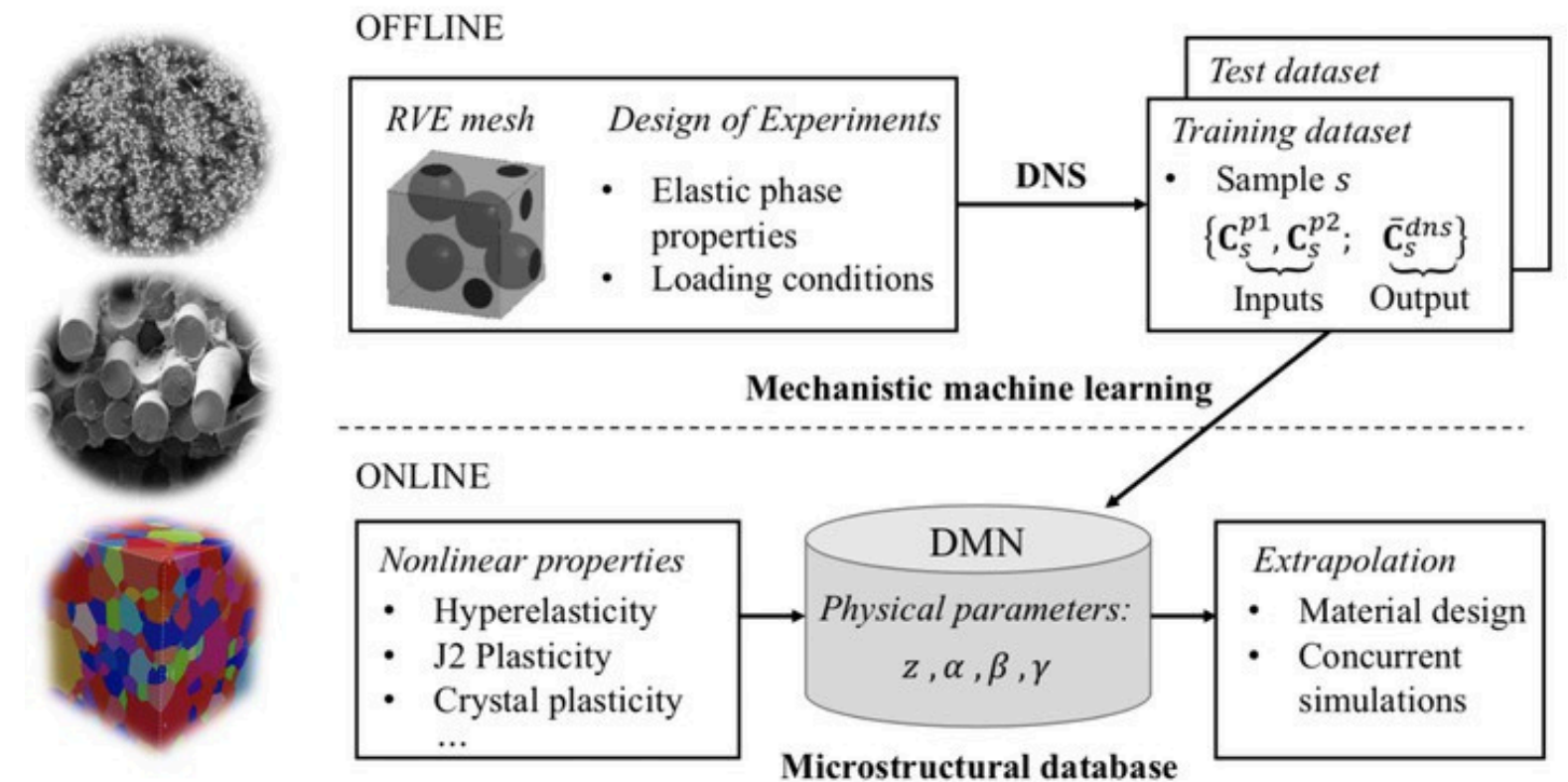
- Multiscale modeling is important for materials with complex microstructures
- Existing methods:
 - Accurate but computationally expensive (DNS, FEM)
 - Efficient but lose physics (micromechanics, Traditional ML Algorithms)
- Need a method that is:
 - Accurate
 - Efficient
 - Physics-consistent

Material Network Concept

- The material network is a data-driven multiscale modeling framework that represents a complex Representative Volume Element (RVE) using a network of simple, connected building blocks
- Inspired by neural networks, but each unit (building block) has a clear physical meaning
- Based on homogenization theory, ensuring physics is preserved
- Provides a reduced representation of heterogeneous materials

Core Idea

- Replace complex microstructures with a hierarchy of simple building blocks
- Each building block has an analytical solution
- The network captures the topology of the RVE
- Reduces computational cost while maintaining accuracy
- Avoids loss of physics seen in traditional machine learning models



THEORY OF MATERIAL NETWORK

Stress-Strain Relation

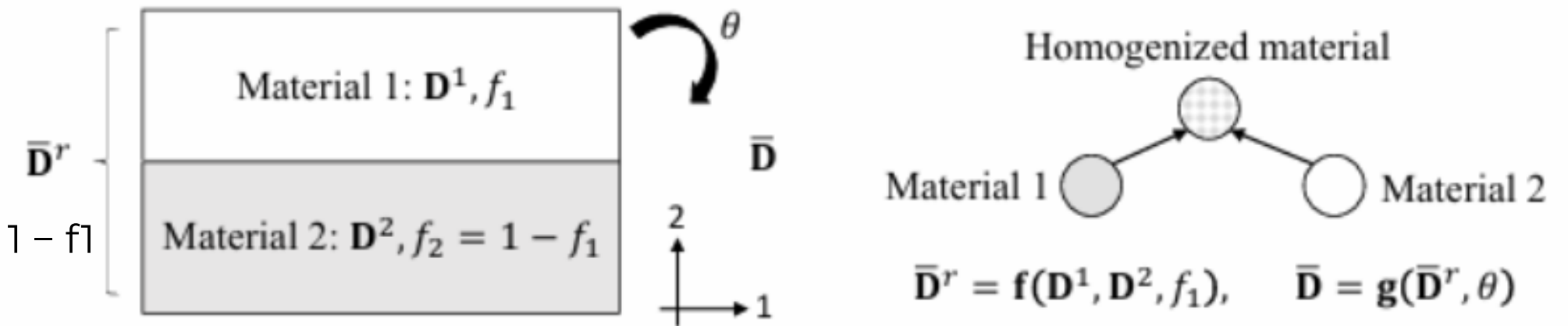
$\bar{\underline{\varepsilon}} = \bar{\underline{\mathbf{D}}} : \bar{\underline{\sigma}}$ Overall Relation

$\underline{\varepsilon}^1 = \underline{\mathbf{D}}^1 : \underline{\sigma}^1, \quad \underline{\varepsilon}^2 = \underline{\mathbf{D}}^2 : \underline{\sigma}^2$ Phase Wise Relation

$$f_1 + f_2 = 1$$

Building Block Structure

- Basic unit: two-layer structure consisting of:
 - Material 1 with compliance matrix $\mathbf{D}^1$ and volume fraction f_1
 - Material 2 with compliance matrix $\mathbf{D}^2$ and volume fraction $f_2 = 1 - f_1$
- Each building block performs two operations:
 - Homogenization
 - Rotation
- These operations transform microscopic properties into effective properties



Homogenization Operation

- Combines two material phases into one equivalent material
- Depends on:
 - Compliance matrices of individual materials
 - Volume fraction
- Derived using:
 - Equilibrium conditions (stress continuity)
 - Kinematic constraints (strain compatibility)
- Produces an intermediate compliance matrix (before rotation)

$$\bar{\mathbf{D}}^r = \mathbf{f}(\mathbf{D}^1, \mathbf{D}^2, f_1)$$

$$\sigma_2^1 = \sigma_2^2 \quad \sigma_3^1 = \sigma_3^2 \quad \varepsilon_1^1 = \varepsilon_1^2$$

Rotation Operation

- Accounts for orientation of material phases
- Uses a rotation matrix to transform the compliance tensor
- Ensures directional dependence of material properties is preserved
- Produces final compliance matrix of the building block

$$\mathbf{R}(\theta) = \begin{Bmatrix} \cos^2 \theta & \sin^2 \theta & \sqrt{2} \sin \theta \cos \theta \\ \sin^2 \theta & \cos^2 \theta & -\sqrt{2} \sin \theta \cos \theta \\ -\sqrt{2} \sin \theta \cos \theta & \sqrt{2} \sin \theta \cos \theta & \cos^2 \theta - \sin^2 \theta \end{Bmatrix}$$

$$\mathbf{R}^{-1}(\theta) = \mathbf{R}(-\theta)$$

$$\bar{\mathbf{D}} = \mathbf{g}(\bar{\mathbf{D}}^r, \theta) = \mathbf{R}(-\theta) \bar{\mathbf{D}}^r \mathbf{R}(\theta)$$

THEORY OF MATERIAL NETWORK

Building Block Function

- Combines homogenization and rotation into a single function
- Maps microscopic properties to effective properties
- Acts as the fundamental unit of the material network

$$\bar{\mathbf{D}} = \mathbf{g} [\mathbf{f}(\mathbf{D}^1, \mathbf{D}^2, f_1), \theta]$$

Importance of Analytical Building Blocks

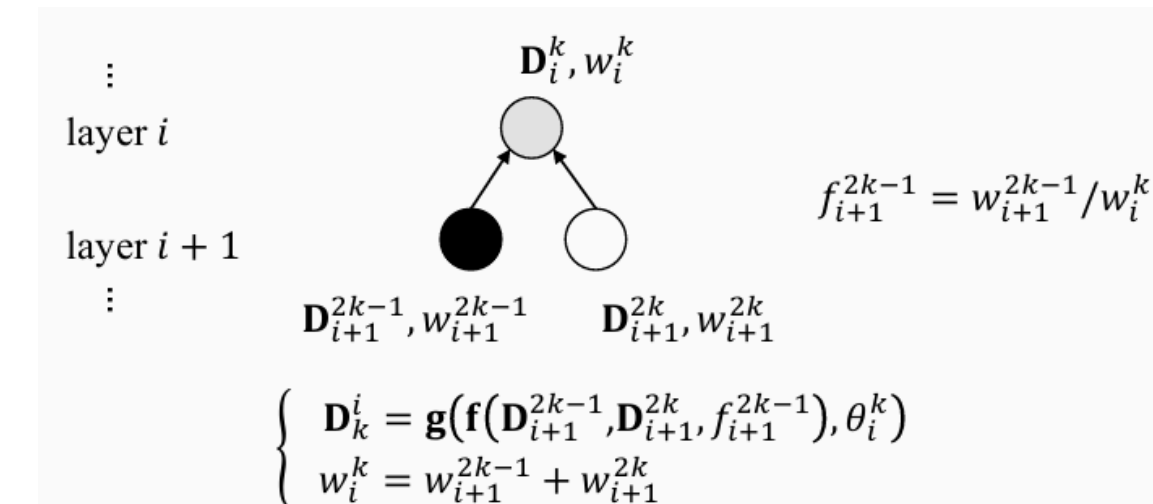
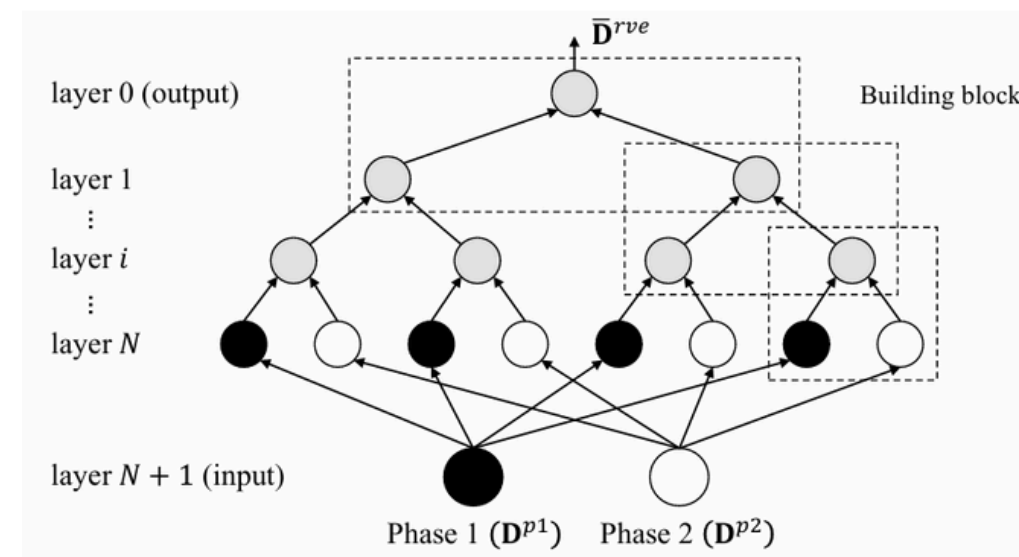
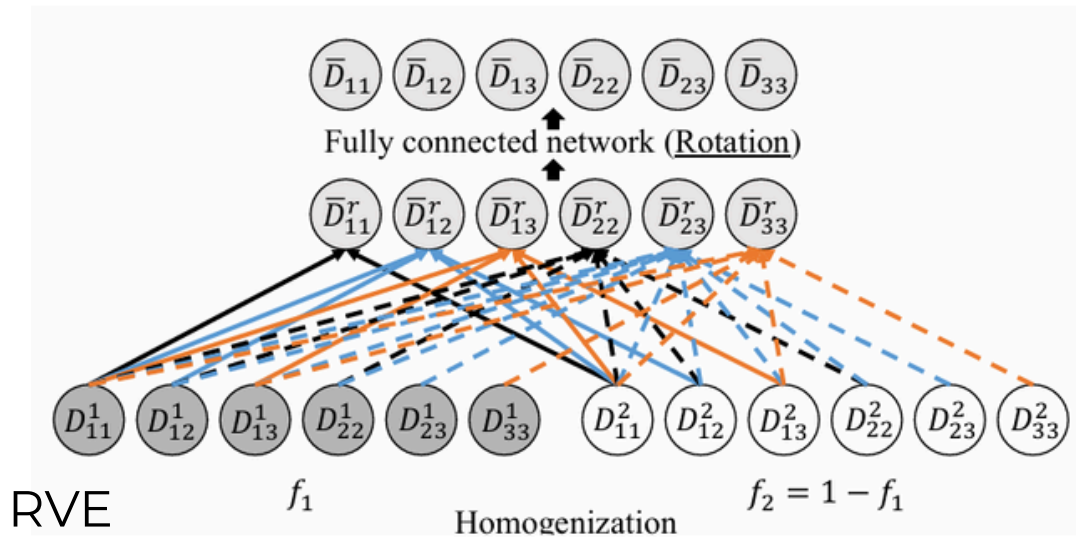
- Analytical solutions avoid approximation errors
- Preserve physical laws such as equilibrium and compatibility
- Enable efficient computation
- Allow use of gradient-based learning methods

Network Architecture

- The material network is organized as a binary tree
- Each node represents a building block
- Layers:
 - Bottom layer: individual material phases
 - Intermediate layers: homogenized combinations
 - Top layer: final macroscopic material
- For depth N:
 - Number of nodes increases as 2^i at layer i

Data Flow in the Network

- Information flows from bottom to top (feedforward)
- At each node:
 - Two child nodes are combined
 - Homogenization and rotation are applied
- Final output represents homogenized RVE properties



Learnable Parameters to be optimized:

- Activations (z) controlling weights
- Rotation angles (theta)
- Network depth (N) determines complexity
- Total number of parameters grows exponentially with depth
- There are total $3 \times 2^N - 1$ fitting parameters

THEORY OF MATERIAL NETWORK

Activation Function (ReLU)

- Weights at the bottom layer are controlled using activation variables
- Rectified Linear Unit (ReLU) is used:
 - $w = \max(z, 0)$
- Automatically removes inactive nodes
- Enables model simplification and compression

$$w_N^j(z_N^j) = Re(z_N^j) = \max(z_N^j, 0). \quad Re'(z_N^j) = \begin{cases} 1 & \text{if } z_N^j > 0 \\ 0 & \text{otherwise} \end{cases}$$

The compliance matrices $\mathbf{D}^r$ and $\mathbf{D}$ at the k -th node in layer i become

$$\mathbf{D}_i^{rk} = \mathbf{f}\left(\mathbf{D}_{i+1}^{2k-1}, \mathbf{D}_{i+1}^{2k}, f_{i+1}^{2k-1}\right)$$

$$\mathbf{D}_i^k = \mathbf{g}\left[\mathbf{f}\left(\mathbf{D}_{i+1}^{2k-1}, \mathbf{D}_{i+1}^{2k}, f_{i+1}^{2k-1}\right), \theta_i^k\right]$$

Homogenized compliance matrix of the RVE can be written as

$$\underbrace{\bar{\mathbf{D}}^{rve}}_{\text{Outputs}} = \mathbf{D}_0^1 = \mathbf{h}\left(\underbrace{\mathbf{D}^{p1}, \mathbf{D}^{p2}}_{\text{Inputs}}, \overbrace{z_N^{j=1,2,\dots,2^N}, \theta_{i=0,1,\dots,N}^{k=1,2,\dots,2^i}}^{\text{Fitting parameters}}\right)$$

Weighting Function

- Introduced to track volume fractions within the network
- Each node has an associated weight
- Parent node weight is sum of child node weights
- Volume fraction is computed using weights
- Ensures correct proportion of material phases

$$w_i^k = \sum_{j=2^{N-i}(k-1)+1}^{2^{N-i}k} w_N^j$$

$$w_i^k = w_{i+1}^{2k-1} + w_{i+1}^{2k}$$

Volume Fraction Representation

- Only bottom-layer weights are independent
- Higher-layer weights are computed recursively
- Volume fractions are derived from weights
- Improves stability during training

$$f_{i+1}^{2k-1} = w_{i+1}^{2k-1} / w_i^k.$$

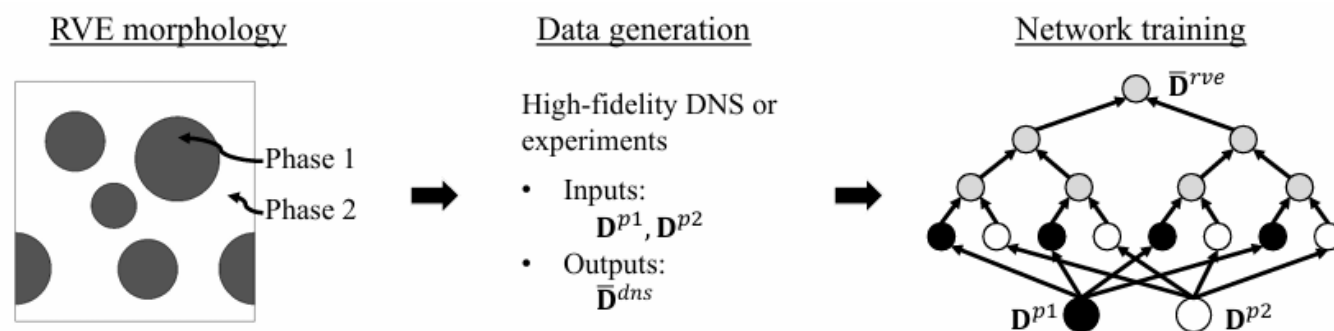
$$\frac{\partial f_{i+1}^{2k-1}}{\partial w_N^j} = \frac{1}{w_i^k} \left(\frac{\partial w_{i+1}^{2k-1}}{\partial w_N^j} - f_{i+1}^{2k-1} \frac{\partial w_i^k}{\partial w_N^j} \right)$$

MACHINE LEARNING OF DEEP MATERIAL NETWORK

Training Overview

- Material network is trained using data from:
 - Direct Numerical Simulation (DNS)
 - Experiments (optional but difficult)
- Goal:
 - Learn optimal network parameters
- Inputs:
 - Material properties of phases ($\mathbf{D}_{p1}$, $\mathbf{D}_{p2}$)
- Output:
 - Homogenized material response ($\mathbf{D}_{dns}$)
- Fitting parameters ($3 \times 2^N - 1$) :

$$z_N^{j=1,2,\dots,2^N}, \theta_{i=0,1,\dots,N}^{k=1,2,\dots,2^i}$$



Regularization Term

- Added to stabilize training
- Prevents extreme parameter values

$$C(z, \theta, \lambda) = C_0(z, \theta) + \lambda L(z)$$

Total cost function

$$L(z) = \left(\sum_i Re(z_N^j) - \xi \right)^2$$

Regularization term

Constraints and Inputs:

- Must ensure physical validity (positive strain energy)
- Materials assumed : Orthoplastic elastic materials

$$\mathbf{D}^{p1} = \begin{Bmatrix} 1/E_{11}^{p1} & -\nu_{12}^{p1}/E_{22}^{p1} \\ & 1/E_{22}^{p1} \end{Bmatrix} \quad \text{and} \quad \mathbf{D}^{p2} = \begin{Bmatrix} 1/E_{11}^{p2} & -\nu_{12}^{p2}/E_{22}^{p2} \\ & 1/E_{22}^{p2} \end{Bmatrix}$$

$$E_{11}^{p1} E_{22}^{p1} = 1, \quad \log_{10}(E_{11}^{p2} E_{22}^{p2}) \in U[-4, 4] \quad \left. \vphantom{\log_{10}(E_{11}^{p2} E_{22}^{p2})} \right\} \text{Removes redundancy using scaling effect}$$

$$\log_{10}(E_{22}^{p1}/E_{11}^{p1}) \in U[-1, 1], \quad \log_{10}(E_{22}^{p2}/E_{11}^{p2}) \in U[-1, 1], \quad \left. \vphantom{\log_{10}(E_{22}^{p2}/E_{11}^{p2})} \right\} \text{These inputs are chosen randomly}$$

$$\frac{G_{12}^{p1}}{\sqrt{E_{22}^{p1} E_{11}^{p1}}} \in U[0.25, 0.5], \quad \frac{G_{12}^{p2}}{\sqrt{E_{22}^{p2} E_{11}^{p2}}} \in U[0.25, 0.5],$$

$$\frac{\nu_{12}^{p1}}{\sqrt{E_{22}^{p1}/E_{11}^{p1}}} \in U[0.3, 0.7], \quad \frac{\nu_{12}^{p2}}{\sqrt{E_{22}^{p2}/E_{11}^{p2}}} \in U[0.3, 0.7] \quad \left. \vphantom{\frac{\nu_{12}^{p2}}{\sqrt{E_{22}^{p2}/E_{11}^{p2}}}} \right\} \text{Poisson's ratios are chosen to ensure positive definite compliance matrices and positive strain energy density.}$$

$$\text{Cost Function: } C_0(z, \theta) = \frac{1}{2N_s} \sum_s C_s(z, \theta) = \frac{1}{2N_s} \sum_s \|\bar{\mathbf{D}}_s^{dns} - \mathbf{h}(\mathbf{D}_s^{p1}, \mathbf{D}_s^{p2}, z, \theta)\|^2 / \|\bar{\mathbf{D}}_s^{dns}\|^2$$

Normalizing error & rotation invariance property:

$$\|\mathbf{D}\| = \sqrt{\text{trace}(\mathbf{D}\mathbf{D})} = \sqrt{\text{trace}(\mathbf{R}^{-1}\mathbf{D}^r\mathbf{R}\mathbf{R}^{-1}\mathbf{D}^r\mathbf{R})} = \sqrt{\text{trace}(\mathbf{R}\mathbf{R}^{-1}\mathbf{D}^r\mathbf{D}^r)} = \|\mathbf{D}^r\|$$

The cost function is normalized using the Frobenius norm to remove scaling effects. This norm is defined as the square root of the sum of squared matrix elements (or trace of $\mathbf{B}^T \mathbf{B}$). Due to trace invariance, it remains unchanged under rotations, making it independent of the reference frame.

MACHINE LEARNING OF DEEP MATERIAL NETWORK

Gradient Computation:

Gradient vector for cost fucntion can be written as:

$$\nabla C = \left(\frac{\partial C}{\partial z_N^j}, \frac{\partial C}{\partial \theta_i^k} \right) = \left(\frac{\partial C_0}{\partial z_N^j} + \lambda \frac{\partial L}{\partial z_N^j}, \frac{\partial C_0}{\partial \theta_i^k} \right)$$
$$j = 1, 2, \dots, 2^N; \quad i = 0, 1, \dots, N; \quad k = 1, 2, \dots, 2^i$$

Due to the simplicity of L, only the gradients of Co are derived. Backpropagation is used to compute gradients and update weights by propagating errors backward through the network, and the same approach is applied to train the material network.

Error Definitions:

Define error terms for each layer

$$\delta_i^n = \frac{\partial C_0}{\partial d_i^n}$$

Measures how sensitive the cost function is to changes in the compliance component (error at node).

$$\alpha_i^n = \frac{\partial C_0}{\partial a_i^n}$$

Measures how sensitive the cost function is to changes in the rotated compliance component.

Output Layer Error:

$$\delta_0^n = \frac{\partial C_0}{\partial d_0^n}$$

Error propagation:

Errors propagated layer by layer

$$\alpha_{i-1}^k = \frac{\partial C_0}{\partial d_{i-1}^j} \frac{\partial d_{i-1}^j}{\partial a_{i-1}^k} = \delta_{i-1}^j \frac{\partial d_{i-1}^j}{\partial a_{i-1}^k}$$
$$\delta_i^n = \frac{\partial C_0}{\partial a_{i-1}^k} \frac{\partial a_{i-1}^k}{\partial d_i^n} = \alpha_{i-1}^k \frac{\partial a_{i-1}^k}{\partial d_i^n}.$$

Gradient w.r.t. Rotation:

Compute sensitivity to rotation parameters

$$\frac{\partial C_0}{\partial \theta_i^k} = \delta_i^j \frac{\partial d_i^j}{\partial \theta_i^k}$$

Gradient w.r.t. Activations:

Compute sensitivity to weights

$$\frac{\partial C_0}{\partial z_N^j} = \left(\alpha_{i-1}^l \frac{\partial a_{i-1}^l}{\partial f_i^m} \frac{\partial f_i^m}{\partial w_N^j} \right) \circ Re'(z_N^j)$$

where $\circ$ denotes the Hadamard (element-wise) product

Initialization of Hyperparameter ξ:

To reduce the influence of the constraint term L(z) during early training, the hyperparameter ξ is initialized as: $E(z_N^{j(0)})2^N = 2^{N-1}$

1. **Input:** Set the initial values for z_N and θ_i ($i = 0, 1, \dots, N$)
2. **Feedforward:** For each layer i , compute a_i and d_i
3. **Output error δ_0 :** Compute the vector δ_0
4. **Backpropagate the error:** For each $i = 1, 2, \dots, N$, compute α_{i-1} and δ_i iteratively
5. **Output:** Compute the gradients of the cost function

Batch Gradient Computation:

Average gradients over dataset

$$\nabla C = \frac{1}{2N_s} \sum_s \nabla C_s$$

Stochastic Gradient Descent:

To speed up training, stochastic gradient descent (SGD) is used. Instead of using all data, gradients are estimated from small randomly selected mini-batches of samples, reducing computation and enabling faster learning.

$$\nabla C \approx \frac{1}{2M} \sum_{i=1}^M \nabla C_{s_i}$$

Parameter Update Rule:

Update parameters using gradient descent

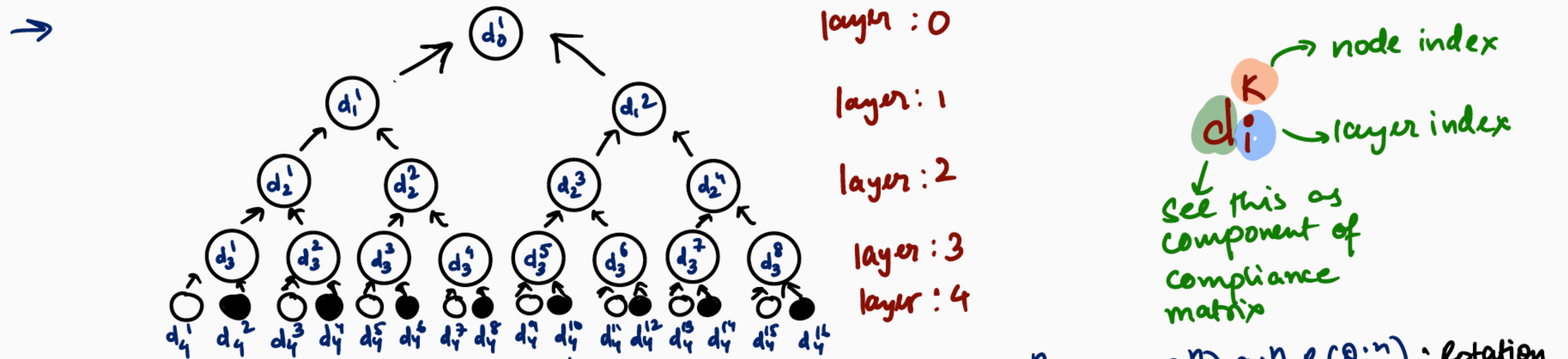
$$z_N^{j'} = z_N^j - \eta \frac{\partial C}{\partial z_N^j} \qquad \theta_i^{k'} = \theta_i^k - \eta \frac{\partial C}{\partial \theta_i^k}$$

Initialization of Parameters:

Parameters initialized randomly

$$z_N^{j(0)} \sim U(0.2, 0.8) \qquad \theta_i^{k(0)} \sim U(-\pi/2, \pi/2)$$

MACHINE LEARNING OF DEEP MATERIAL NETWORK



At each node: $a_i^n = f(d_{i+1}^{2n-1}, d_{i+1}^{2n})$: Homogenization ; $d_i^n = R(-\theta_i^n) a_i^n R(\theta_i^n)$: Rotation
(n is node index in the below eq. as per the paper)
 a_i^n : intermediate out put (before rotation) , d_i^n : final output of node.

→ Loss function : $C \equiv$ difference between d_0^1 (final output) and actual output (our dataset).

→ Output error : $\delta_0^1 = \frac{\partial C}{\partial d_0^1}$ (how much loss changes if final output changes)

LOSS DEPENDS ONLY ON THE FINAL OUTPUT!!!

MACHINE LEARNING OF DEEP MATERIAL NETWORK

Why we define errors at each node?

$C \rightarrow$ function of d_0^1 , but d_0^1 depends on d_1^1, d_1^2

this depends on $d_2^1, d_2^2 \dots$

this depends on $d_3^1, d_3^2 \dots$

which ultimately depends on Z and θ .

So, we want $\partial C / \partial z$ and $\partial C / \partial \theta$

- z and θ are deep inside network
- not directly connected to output

Solution: Chain Rule:

$$\partial C / \partial z = \partial C / \partial d_0 \times \partial d_0 / \partial d_1 \times \partial d_1 / \partial d_2 \times \dots \times \partial (-) / \partial z$$

(similarly for θ)

Computing this directly is impossible

So, we define $\delta = \partial C / \partial d$
(error at node output)

, $\alpha = \partial C / \partial a$
(error before rotation)

↓
exponential computation.
(back propagation converts exponential computation into linear computation)

MACHINE LEARNING OF DEEP MATERIAL NETWORK

$$\delta_i^n = \frac{\partial C}{\partial d_i^n}, \quad \alpha_i^n = \frac{\partial C}{\partial a_i^n}$$

Error Propagation:

$$\alpha_{i-1}^k = \frac{\partial C}{\partial d_{i-1}^j} \frac{\partial d_{i-1}^j}{\partial a_{i-1}^k} = \delta_{i-1}^j \frac{\partial d_{i-1}^j}{\partial a_{i-1}^k} \quad ; \quad \delta_i^n = \frac{\partial C}{\partial a_{i-1}^k} \frac{\partial a_{i-1}^k}{\partial d_i^n} = \alpha_{i-1}^k \frac{\partial a_{i-1}^k}{\partial d_i^n}$$

→ Now we can compute error δ_i and α_i at any layer in network.

- Within one step of back propagation, error of cost function propagates backward from layer $i=0$ to layer $i=N$ (bottom).
- After backpropagation of error for compliance matrices, we can compute rate of change of the cost function with respect to rotation angle θ_i^k by:

$$\frac{\partial C}{\partial \theta_i^k} = \delta_i^j \frac{\partial d_i^j}{\partial \theta_i^k}$$

MACHINE LEARNING OF DEEP MATERIAL NETWORK

- Rate of change of cost function wrt. activation z_N^j (activation only at bottom layer):

$$\frac{\partial C}{\partial z_N^j} = \left(\alpha_{i-1}^l \frac{\partial a_{i-1}^l}{\partial f_i^m} \frac{\partial f_i^m}{\partial w_N^j} \right) \circ \text{Re}'(z_N^j)$$

And then we can update our parameters simply how we do in any gradient descent;

$$z_N^{j'} = z_N^j - \eta \frac{\partial C}{\partial z_N^j} \quad ; \quad \theta_i^{k'} = \theta_i^k - \eta \frac{\partial C}{\partial \theta_i^k}$$

MODEL COMPRESSION AND PARAMETER REDUCTION

Model Compression

- Reduces network complexity
- Improves efficiency and generalization

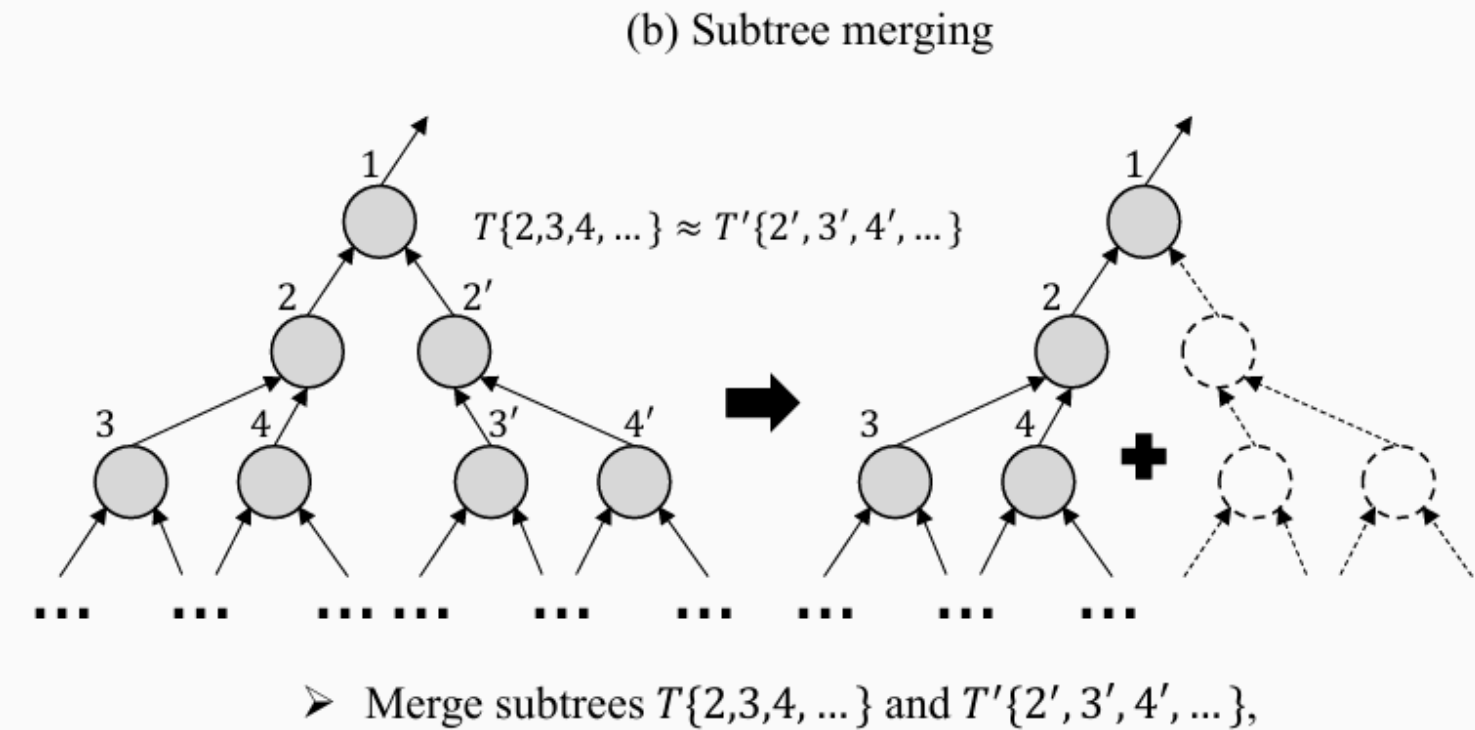
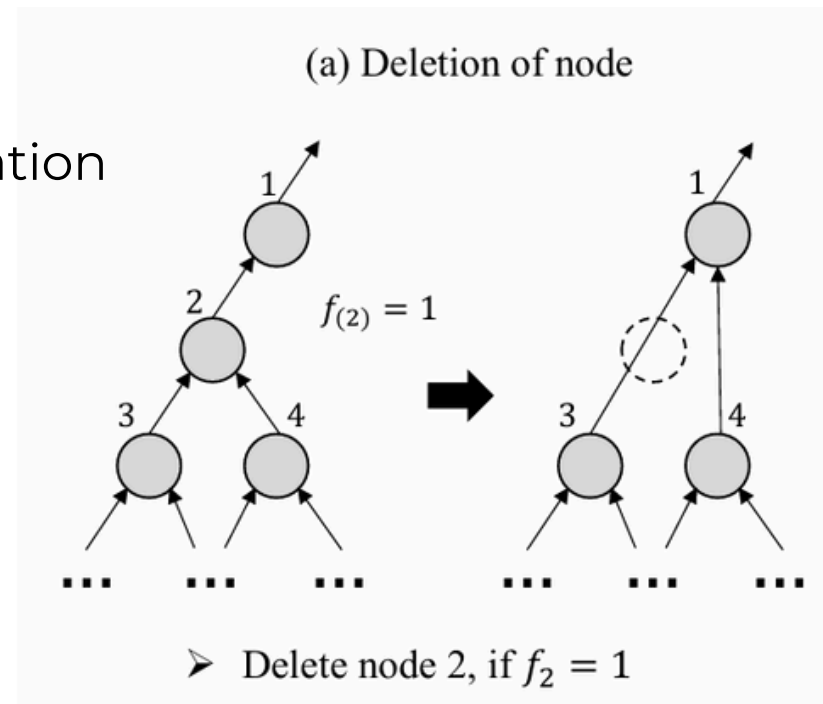
Compression Techniques

1. Node Deletion

- Remove nodes with weight = 1
- Simplifies network

2. Subtree Merging

- Merge similar substructures
- Reduces redundancy



Parameter Update after Compression

Taking into account the rotation effect of node with $f=1$:

$$\theta_{(1)}^{new} = \theta_{(1)}^{old} + \theta_{(2)}^{old}$$

Similarity search for merging

$$\delta_{T-T'}^f = \max_{m=3} (|f_{(m)} - f_{(m')}|) \quad \text{and} \quad \delta_{T-T'}^\theta = \max_{m=2} \left(\frac{1}{\pi} |\text{mod}(\theta_{(m)} - \theta_{(m')}, \pi)| \right)$$

Adjust parameters after merging

$$w_{(m)}^{new} = w_{(m)}^{old} + w_{(m')}^{old}, \quad \theta_{(m)}^{new} = \frac{1}{2}(\theta_{(m)}^{old} + \theta_{(m')}^{old}) \quad \text{for } m \geq 2.$$

Ordered Material Network Condition

- Before merging, the network must be ordered
- Condition at each layer i :

$$w_i^{2k-1} \geq w_i^{2k} \quad \forall k \in [1, 2^{i-1}]$$

Before performing similarity search and subtree merging, the material network is reordered using weighting functions. To reduce training time, network compression is carried out once every 10 epochs.

CODE IMPLEMENTATION AND DATA SET

1. Load the dataset containing **inputs as material properties of two phases** and **outputs as homogenized material response**, then **initialize trainable parameters** (weights and rotation angles).
2. Write a **forward function** that takes the inputs and parameters, builds the **binary tree**, performs homogenization followed by rotation at each node, and outputs the final predicted material response.
3. Define a **loss function** that measures the difference between predicted output and true output from the dataset, and include a **regularization** term to keep parameters stable.
4. Compute **gradients of the loss with respect to all parameters using backpropagation** so that you know how each parameter affects the error.
5. Run a **training loop** where for **each batch you perform forward pass, compute loss, calculate gradients, update parameters using gradient descent**, and finally **apply model compression** to simplify the network.

Dataset Format (DNS-Based Data):

The dataset follows the format obtained using the Direct Numerical Simulation (DNS) method, where:

Input Columns (12)

Phase 1:

D11_p1, D12_p1, D13_p1, D22_p1, D23_p1, D33_p1

Phase 2:

D11_p2, D12_p2, D13_p2, D22_p2, D23_p2, D33_p2

Output Columns (6)

Homogenized Properties:

D11_hom, D12_hom, D13_hom, D22_hom, D23_hom, D33_hom

Note:

This dataset represents material behavior computed using DNS, where phase-wise compliance matrices are used as inputs to predict the effective (homogenized) compliance matrix.

References:

1. [Zeliang Liu's DMN Blog](#)
2. Liu, Zeliang, C. T. Wu, and M. Koishi. 2019. "A Deep Material Network for Multiscale Topology Learning and Accelerated Nonlinear Modeling of Heterogeneous Materials."

Code Cell 1 & 2 : Library Import & DNS Data Upload

What this block does

This block imports the required libraries, checks whether GPU is available, sets random seeds for reproducibility, and uploads the DNS .mat file so the data can be loaded into Python.

How it works

- Imports the required libraries for neural network training, numerical computation, visualization, and .mat file handling.
- Detects and selects GPU (CUDA) if available, otherwise uses CPU, and displays device information.
- Sets PyTorch and NumPy random seeds to ensure reproducible results.
- Uploads and loads the DNS .mat dataset into the variable data.
- Displays the uploaded file name and the total number of samples available in C_eff_data.

Code Cell 1 & 2 : Library Import & DNS Data Upload

Cell 1: Import Libraries with GPU Detection

=====

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
import numpy as np
import matplotlib.pyplot as plt
import scipy.io as sio
from google.colab import files
import time
import warnings
warnings.filterwarnings('ignore')

# Check GPU
if torch.cuda.is_available():
    device = torch.device('cuda')
    print(f"✓ GPU Available: {torch.cuda.get_device_name(0)}")
    print(f"✓ GPU Memory: {torch.cuda.get_device_properties(0).total_memory / 1e9:.1f} GB")
else:
    device = torch.device('cpu')
    print("✓ Using CPU (Go to Runtime → Change runtime type → T4 GPU for faster training)")

print(f"✓ PyTorch version: {torch.__version__}")

torch.manual_seed(42)
np.random.seed(42)
```

-
- ✓ GPU Available: Tesla T4
 - ✓ GPU Memory: 15.6 GB
 - ✓ PyTorch version: 2.10.0+cu128
-

=====

Cell 2: Upload DNS Data

=====

```
print("Upload your .mat file")
uploaded = files.upload()
file_name = list(uploaded.keys())[0]
data = sio.loadmat(file_name)

print(f"✓ Loaded: {file_name}")
print(f"Samples: {data['C_eff_data'].shape[2]}")
```

Upload your .mat file

No file chosen

Upload widget is only availab

Saving C_eff_dataset_500.mat to C_eff_dataset_500.mat

✓ Loaded: C_eff_dataset_500.mat

Samples: 500

Code Cell 3 : Rotation Functions- GPU Optimized

What this block does

This block defines GPU-optimized rotation and matrix-conversion functions used to transform compliance tensors in Mandel notation during neural network training.

How it works

$$\mathbf{R}(\theta) = \begin{Bmatrix} \cos^2 \theta & \sin^2 \theta & \sqrt{2} \sin \theta \cos \theta \\ \sin^2 \theta & \cos^2 \theta & -\sqrt{2} \sin \theta \cos \theta \\ -\sqrt{2} \sin \theta \cos \theta & \sqrt{2} \sin \theta \cos \theta & \cos^2 \theta - \sin^2 \theta \end{Bmatrix}$$

- Defines the rotation matrix in Mandel notation using trigonometric relations from Eq. 9.
- Creates batch-wise rotation matrices for efficient GPU computation.
- Converts 3×3 symmetric Mandel compliance matrices into compact 6-component vectors.
- Converts 6-component Mandel vectors back into full 3×3 symmetric matrices.
- Applies tensor rotation using the relation:

$$\bar{\mathbf{D}} = \mathbf{g}(\bar{\mathbf{D}}^r, \theta) = \mathbf{R}(-\theta) \bar{\mathbf{D}}^r \mathbf{R}(\theta)$$

- Uses batch matrix multiplication (torch.bmm) for fast parallel tensor operations on GPU.
- Returns the rotated compliance tensors back in compact vector form.

Code Cell 3 : Rotation Functions- GPU Optimized

```
# =====
# Cell 3: Rotation Functions - GPU Optimized (CORRECTED)
# =====

def rotation_matrix_torch(theta, batch_size=None):
    """
    Eq. 9 - Rotation matrix in Mandel notation
    For 3x3 matrix (2D plane strain)
    """
    if batch_size is None:
        batch_size = theta.shape[0]

    device = theta.device
    dtype = theta.dtype

    c = torch.cos(theta)
    s = torch.sin(theta)
    sqrt2 = np.sqrt(2)

    # Create rotation matrix for each sample in batch
    R = torch.zeros(batch_size, 3, 3, device=device, dtype=dtype)

    # Fill according to Eq. 9
    R[:, 0, 0] = c**2
    R[:, 0, 1] = s**2
    R[:, 0, 2] = sqrt2 * c * s

    R[:, 1, 0] = s**2
    R[:, 1, 1] = c**2
    R[:, 1, 2] = -sqrt2 * c * s

    R[:, 2, 0] = -sqrt2 * c * s
    R[:, 2, 1] = sqrt2 * c * s
    R[:, 2, 2] = c**2 - s**2 # cos^2θ - sin^2θ = cos(2θ)

    return R
```

```
def mat3_to_vec6(D_mat):
    """
    Convert 3x3 Mandel compliance matrix to 6-component vector
    [D11, D12, D13, D22, D23, D33]
    """
    batch_size = D_mat.shape[0]
    device = D_mat.device
    dtype = D_mat.dtype

    D_vec = torch.zeros(batch_size, 6, device=device, dtype=dtype)
    D_vec[:, 0] = D_mat[:, 0, 0] # D11
    D_vec[:, 1] = D_mat[:, 0, 1] # D12
    D_vec[:, 2] = D_mat[:, 0, 2] # D13
    D_vec[:, 3] = D_mat[:, 1, 1] # D22
    D_vec[:, 4] = D_mat[:, 1, 2] # D23
    D_vec[:, 5] = D_mat[:, 2, 2] # D33
    return D_vec

def vec6_to_mat3(D_vec):
    """
    Convert 6-component vector to 3x3 Mandel compliance matrix
    """
    batch_size = D_vec.shape[0]
    device = D_vec.device
    dtype = D_vec.dtype

    D_mat = torch.zeros(batch_size, 3, 3, device=device, dtype=dtype)
    D_mat[:, 0, 0] = D_vec[:, 0] # D11
    D_mat[:, 0, 1] = D_vec[:, 1] # D12
    D_mat[:, 0, 2] = D_vec[:, 2] # D13
    D_mat[:, 1, 0] = D_vec[:, 1] # D12 (symmetric)
    D_mat[:, 1, 1] = D_vec[:, 3] # D22
    D_mat[:, 1, 2] = D_vec[:, 4] # D23
    D_mat[:, 2, 0] = D_vec[:, 2] # D13 (symmetric)
    D_mat[:, 2, 1] = D_vec[:, 4] # D23 (symmetric)
    D_mat[:, 2, 2] = D_vec[:, 5] # D33
    return D_mat
```

$$R(\theta) = \begin{bmatrix} \cos^2 \theta & \sin^2 \theta & \sqrt{2} \cos \theta \sin \theta \\ \sin^2 \theta & \cos^2 \theta & -\sqrt{2} \cos \theta \sin \theta \\ -\sqrt{2} \cos \theta \sin \theta & \sqrt{2} \cos \theta \sin \theta & \cos^2 \theta - \sin^2 \theta \end{bmatrix}$$

$$D = \begin{bmatrix} D_{11} & D_{12} & D_{13} \\ D_{12} & D_{22} & D_{23} \\ D_{13} & D_{23} & D_{33} \end{bmatrix}$$

$$[D_{11}, D_{12}, D_{13}, D_{22}, D_{23}, D_{33}]$$

$$D_{rot} = R(-\theta)DR(\theta)$$

```
def rotate_compliance_batch(D_vec, theta):
    """
    Rotate batch of compliance vectors using Eq. 11
    D_bar = R(-θ) * D^r * R(θ)
    """
    batch_size = D_vec.shape[0]

    # Convert to matrix form
    D_mat = vec6_to_mat3(D_vec)

    # Get rotation matrices
    R = rotation_matrix_torch(theta, batch_size)
    R_minus = rotation_matrix_torch(-theta, batch_size)

    # Apply rotation: R(-θ) * D * R(θ)
    # batch matrix multiplication
    D_rotated = torch.bmm(R_minus, torch.bmm(D_mat, R))

    # Convert back to vector form
    return mat3_to_vec6(D_rotated)
```

```
print("✓ GPU-optimized rotation functions defined (CORRECTED)")
```

```
✓ GPU-optimized rotation functions defined (CORRECTED)
```

Code Cell 4 : Building Block Module

What this block does

This block defines the BuildingBlock neural network module, which combines two material compliance tensors using homogenization and then rotates the result.

How it works

- Takes compliance vectors of material 1 and material 2 as input.
- Accepts volume fraction f_1 and rotation angle θ as external inputs.
- Converts scalar inputs into batch-sized tensors if needed.
- Computes $f_2 = 1 - f_1$.
- Unpacks the 6 Mandel compliance components from both materials.
- Applies the two-layer homogenization formula to compute the intermediate compliance tensor D_{prime} .
- Stacks the homogenized components back into a 6-component vector.
- Rotates the homogenized compliance tensor using the previously defined rotation function.
- Returns the rotated compliance vector as the output.

Code Cell 4 : Building Block Module

```
# =====
# Cell 4: Building Block Module (CORRECTED)
# =====

class BuildingBlock(nn.Module):
    """
    Implements the two-layer building block described in Section 2.1
    """
    def __init__(self):
        super().__init__()
        # f1 will be set externally (volume fraction of material 1)
        # theta will be set externally (rotation angle)

    def forward(self, D1, D2, f1, theta):
        """
        Forward pass implementing Eq. 8 (homogenization) and Eq. 11 (rotation)

        Args:
            D1: Compliance vector of material 1 [batch, 6]
            D2: Compliance vector of material 2 [batch, 6]
            f1: Volume fraction of material 1 [batch] or scalar
            theta: Rotation angle [batch] or scalar
        """
        batch_size = D1.shape[0]
        device = D1.device
        dtype = D1.dtype
```

```
        # Ensure f1 and theta are tensors with batch dimension
        if not isinstance(f1, torch.Tensor):
            f1 = torch.tensor(f1, device=device, dtype=dtype)
        if f1.dim() == 0:
            f1 = f1.expand(batch_size)

        if not isinstance(theta, torch.Tensor):
            theta = torch.tensor(theta, device=device, dtype=dtype)
        if theta.dim() == 0:
            theta = theta.expand(batch_size)

        f2 = 1 - f1

        # Unpack compliance components
        D11_1, D12_1, D13_1, D22_1, D23_1, D33_1 = (
            D1[:, 0], D1[:, 1], D1[:, 2], D1[:, 3], D1[:, 4], D1[:,
        )
        D11_2, D12_2, D13_2, D22_2, D23_2, D33_2 = (
            D2[:, 0], D2[:, 1], D2[:, 2], D2[:, 3], D2[:, 4], D2[:,
        )
```

```
        # Eq. 8 - Homogenization of two-layer structure
        Gamma = f1 * D11_2 + f2 * D11_1 + 1e-10

        D11_prime = (D11_1 * D11_2) / Gamma
        D12_prime = (f1 * D12_1 * D11_2 + f2 * D12_2 * D11_1) / Gamma
        D13_prime = (f1 * D13_1 * D11_2 + f2 * D13_2 * D11_1) / Gamma

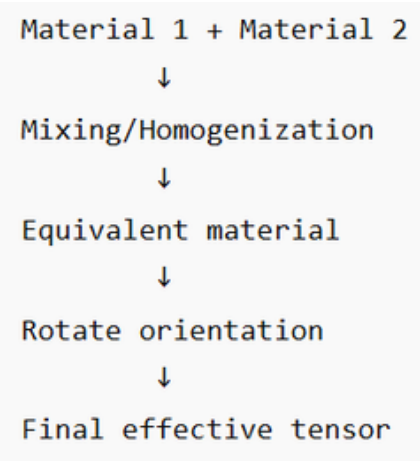
        D22_prime = (f1 * D22_1 + f2 * D22_2 -
                     (f1 * f2 * (D12_1 - D12_2)**2) / Gamma)
        D23_prime = (f1 * D23_1 + f2 * D23_2 -
                     (f1 * f2 * (D13_1 - D13_2) * (D12_1 - D12_2)) / Gamma)
        D33_prime = (f1 * D33_1 + f2 * D33_2 -
                     (f1 * f2 * (D13_1 - D13_2)**2) / Gamma)

        # Stack into vector
        D_prime = torch.stack([
            D11_prime, D12_prime, D13_prime,
            D22_prime, D23_prime, D33_prime
        ], dim=1)

        # Eq. 11 - Rotation
        D_rotated = rotate_compliance_batch(D_prime, theta)

        return D_rotated

print("✓ BuildingBlock defined (CORRECTED)")
```



Equilibrium condition

$$\sigma_2^1 = \sigma_2^2, \quad \sigma_3^1 = \sigma_3^2$$

Kinematic Constraint

$$\varepsilon_1^1 = \varepsilon_1^2$$

Equation 8

$$\bar{D}_{11}^r = \frac{1}{\Gamma} (D_{11}^1 D_{11}^2),$$

$$\bar{D}_{12}^r = \frac{1}{\Gamma} (f_1 D_{12}^1 D_{11}^2 + f_2 D_{12}^2 D_{11}^1),$$

$$\bar{D}_{13}^r = \frac{1}{\Gamma} (f_1 D_{13}^1 D_{11}^2 + f_2 D_{13}^2 D_{11}^1),$$

$$\bar{D}_{22}^r = f_1 D_{22}^1 + f_2 D_{22}^2 - \frac{1}{\Gamma} f_1 f_2 (D_{12}^1 - D_{12}^2)^2,$$

$$\bar{D}_{23}^r = f_1 D_{23}^1 + f_2 D_{23}^2 - \frac{1}{\Gamma} f_1 f_2 (D_{13}^1 - D_{13}^2)(D_{12}^1 - D_{12}^2),$$

$$\bar{D}_{33}^r = f_1 D_{33}^1 + f_2 D_{33}^2 - \frac{1}{\Gamma} f_1 f_2 (D_{13}^1 - D_{13}^2)^2,$$

$$\Gamma = f_1 D_{11}^2 + f_2 D_{11}^1 \quad \text{and} \quad f_2 = 1 - f_1$$

$$\bar{\mathbf{D}}^r = \mathbf{f}(\mathbf{D}^1, \mathbf{D}^2, f_1).$$

Equation 11

$$\bar{\mathbf{D}} = \mathbf{g}(\bar{\mathbf{D}}^r, \theta) = \mathbf{R}(-\theta) \bar{\mathbf{D}}^r \mathbf{R}(\theta)$$

Code Cell 5 : Material Network

What this block does

This block defines the Deep Material Network (DMN) architecture using a hierarchical binary-tree structure to homogenize and predict effective material compliance tensors.

How it works

- Defines the MaterialNetwork class as a Deep Material Network (DMN) with a hierarchical binary-tree architecture for predicting effective material compliance tensors.
- Initializes the network depth, computes the number of bottom-layer nodes (2^N), creates learnable bottom-layer activation weights using ReLU activation, and defines learnable rotation angles for each node in every layer.
- Stores the reference compliance tensors for both material phases and uses the previously defined BuildingBlock module for homogenization and tensor rotation operations.
- Computes bottom-layer node weights using Eq. 21, assigns weighted phase compliances to bottom nodes, and performs hierarchical homogenization recursively from the bottom layer to the top output node.
$$w_N^j(z_N^j) = \text{Re}(z_N^j) = \max(z_N^j, 0)$$
- Calculates local volume tractions and applies homogenization and rotation at every parent node using the building block to generate the final effective compliance tensor.
- Includes a regularization term to constrain activation weights and stabilize training.

Code Cell 5 : Material Network

```
# =====
# Cell 5: Material Network (CORRECTED)
# =====

class MaterialNetwork(nn.Module):
    """
    Deep Material Network as described in Section 2.2
    Binary tree structure with depth N
    """
    def __init__(self, depth, D1_init, D2_init):
        """
        Args:
            depth: Depth of the network (N)
            D1_init: Initial compliance vector for phase 1 [6]
            D2_init: Initial compliance vector for phase 2 [6]
        """
        super().__init__()
        self.depth = depth
        self.n_bottom = 2**depth # Number of nodes at bottom layer

        # Learnable activation parameters at bottom layer (Eq. 21)
        self.z = nn.Parameter(torch.rand(self.n_bottom) * 0.5 + 0.3) # Initialize between 0.3 and 0.8

        # Learnable rotation angles for each node
        # theta_i^k for layer i, node k
        self.thetas = nn.ParameterList()
        for i in range(depth + 1): # layers 0 to N
            n_nodes = 2**i
            self.thetas.append(
                nn.Parameter(torch.rand(n_nodes) * np.pi - np.pi/2) # Initialize between -pi/2 and pi/2
            )

        # Store reference compliance for both phases
        self.register_buffer('D1_ref', D1_init.unsqueeze(0)) # [1, 6]
        self.register_buffer('D2_ref', D2_init.unsqueeze(0)) # [1, 6]

        # Create building blocks
        self.building_block = BuildingBlock()

    def get_bottom_weights(self, batch_size):
        """
        Get weights at bottom layer using ReLU activation (Eq. 21)
        w_N^j = ReLU(z_N^j)
        """
        return torch.relu(self.z).unsqueeze(0).expand(batch_size, -1) # [batch, n_bottom]
```

Equation 21

$$w_N^j(z_N^j) = Re(z_N^j) = \max(z_N^j, 0)$$

The class sets the tree depth, the number of bottom nodes as 2**depth, learns bottom activation values z, learns a rotation angle for each node, stores fixed reference compliance tensors as buffers, and plugs in the BuildingBlock from the previous cell.

Buffer Unsqueeze

Before:

```
[1 2 3 4 5 6]
```

After:

```
[
  [1 2 3 4 5 6]
]
```

Code Cell 5 : Material Network

```
def compute_node_compliance(self, D1_batch, D2_batch, node_weights, layer, batch_size):
    """
    Compute compliance for all nodes at a given layer using hierarchical homogenization
    """

    n_nodes = 2**layer
    node_comp = []

    for node in range(n_nodes):
        if layer == self.depth:
            # Bottom layer: each node gets mixture of phases
            # The node's compliance is a weighted mixture based on its phase assignment
            # For simplicity, we alternate phase assignments initially
            # This will be learned through training
            if node % 2 == 0:
                node_comp.append(D1_batch * node_weights[:, node:node+1])
            else:
                node_comp.append(D2_batch * node_weights[:, node:node+1])
        else:
            # Higher layers: homogenize child nodes
            child1_idx = 2 * node
            child2_idx = 2 * node + 1

            D_child1 = node_comp[child1_idx]
            D_child2 = node_comp[child2_idx]

            # Get volume fractions for children
            w_child1 = node_weights[:, 2*node*2**(self.depth-layer):(2*node+1)*2**(self.depth-layer)].sum(dim=1)
            w_child2 = node_weights[:, (2*node+1)*2**(self.depth-layer):(2*node+2)*2**(self.depth-layer)].sum(dim=1)
            w_total = w_child1 + w_child2 + 1e-10
            f1 = w_child1 / w_total

            # Get rotation angle for this node
            theta = self.thetas[layer][node].expand(batch_size)

            # Homogenize and rotate
            D_parent = self.building_block(D_child1, D_child2, f1, theta)
            node_comp.append(D_parent)

    return node_comp
```

This function takes the material compliances and learned weights, finds the child nodes for each parent in the hierarchy, computes the local phase fraction, applies the rotation angle, and returns the homogenized compliance for that layer

What compute_node_compliance does

This function manages the ENTIRE layer.

So if it is working on a layer with 8 nodes, it:

- loops through all 8 nodes,
- gets their compliances,
- organizes which nodes combine together,
- computes local f1,
- gets theta,
- calls BuildingBlock.

So this function is like the manager/coordinator.

What BuildingBlock does

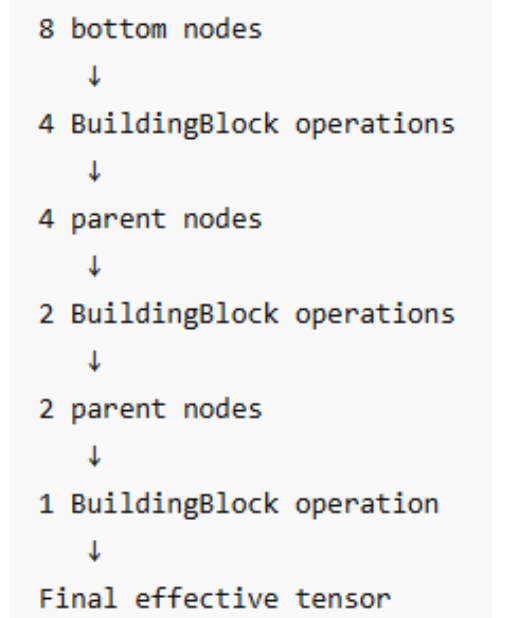
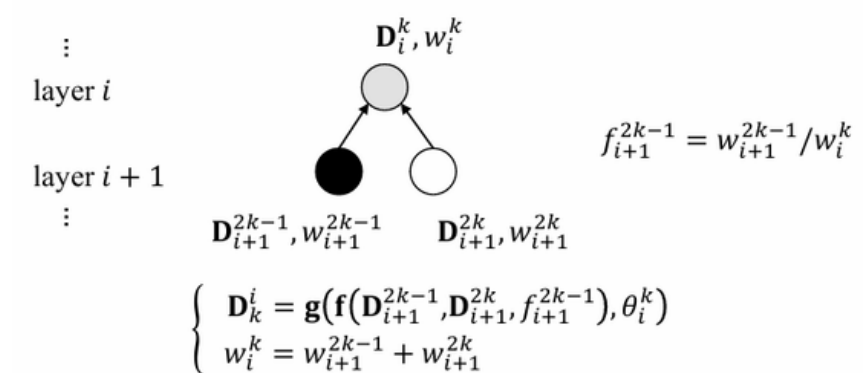
BuildingBlock only works on ONE PAIR at a time.

$$w_i^k = \sum_{j=2^{N-i}(k-1)+1}^{2^{N-i}k} w_N^j$$

$$w_i^k = w_{i+1}^{2k-1} + w_{i+1}^{2k}$$

$$f_{i+1}^{2k-1} = w_{i+1}^{2k-1} / w_i^k.$$

w_i^k be the weight of the node at layer i and node k.



Code Cell 5 : Material Network

```
def forward(self, D1, D2):
    """
    Forward pass through the material network
    Args:
        D1: Compliance vectors for phase 1 [batch, 6]
        D2: Compliance vectors for phase 2 [batch, 6]
    Returns:
        Homogenized compliance at output layer [batch, 6]
    """
    batch_size = D1.shape[0]

    # Get bottom layer weights (Eq. 21)
    w_bottom = self.get_bottom_weights(batch_size) # [batch, n_bottom]

    # Compute compliances for all layers
    all_compliances = []

    # For simplicity, we'll implement a recursive-like approach
    # Start from bottom and work up

    # Bottom layer nodes
    n_bottom = self.n_bottom
    bottom_compliances = []
    for j in range(n_bottom):
        # Each bottom node gets a weighted mixture of both phases
        # The weight determines which phase dominates
        if j % 2 == 0:
            bottom_compliances.append(D1.clone() * w_bottom[:, j:j+1])
        else:
            bottom_compliances.append(D2.clone() * w_bottom[:, j:j+1])

    # Work upward through layers
    current_compliances = bottom_compliances

    for layer in range(self.depth - 1, -1, -1):
        n_nodes = 2**layer
        next_compliances = []

        for node in range(n_nodes):
            child1 = 2 * node
            child2 = 2 * node + 1

            D_child1 = current_compliances[child1]
            D_child2 = current_compliances[child2]

            # Compute volume fractions
            descendant_start = node * 2**(self.depth - layer)
            descendant_end = (node + 1) * 2**(self.depth - layer)

            w_subtree = w_bottom[:, descendant_start:descendant_end]
            w_total = w_subtree.sum(dim=1) + 1e-10
```

```
        # Split into left and right children
        mid = (descendant_start + descendant_end) // 2
        w_left = w_bottom[:, descendant_start:mid].sum(dim=1)
        w_right = w_bottom[:, mid:descendant_end].sum(dim=1)

        f1 = w_left / w_total

        # Get rotation angle
        theta = self.thetas[layer][node].expand(batch_size)

        # Homogenize
        D_parent = self.building_block(D_child1, D_child2, f1, theta)
        next_compliances.append(D_parent)
```

current_compliances = next_compliances

```
# Output layer (layer 0) has 1 node
return current_compliances[0]
```

What forward() does

The forward() function performs the complete end-to-end hierarchical material-network computation. It starts from the weighted bottom-layer compliances, moves upward through all layers of the binary tree, computes local volume fractions f_1 and rotation angles θ , repeatedly calls the BuildingBlock for homogenization and tensor rotation, and finally returns the single effective homogenized compliance tensor at the root node.

λ_{reg} controls the strength of the regularization penalty, determining how strongly the model enforces the weight constraint. The factor 0.5 is included for mathematical convenience because it simplifies the gradient during backpropagation when differentiating the squared error term.

```
self.depth = 3
layer = 1
node = 0
```

$$\begin{aligned} \text{start} &= 0 \times 2^{(3-1)} & \text{end} &= (0+1) \times 2^{(3-1)} \\ &= 0 \times 2^2 & &= 1 \times 2^2 \\ &= 0 \times 4 & &= 1 \times 4 \\ &= 0 & &= 4 \end{aligned}$$

0 1 2 3

```
w_subtree = w_bottom[:, 0:4]
```

```
w_bottom = [0.10, 0.20, 0.15, 0.25, 0.05, 0.08, 0.10, 0.07]
```

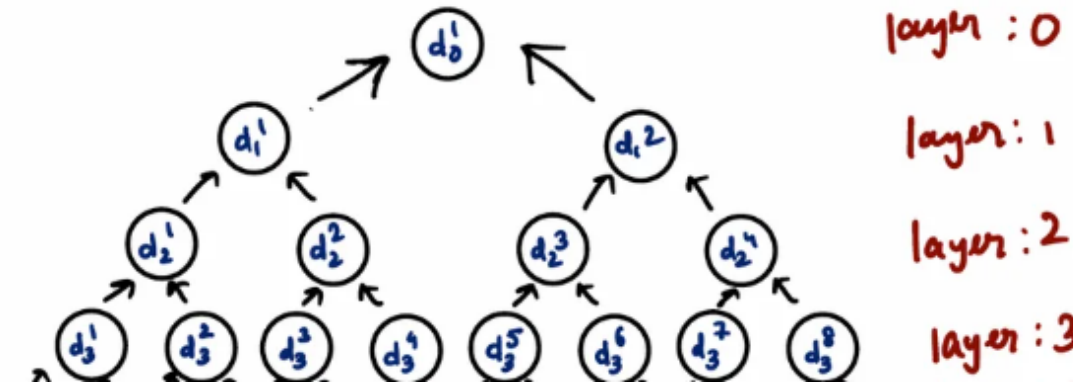
```
w_subtree = [0.10, 0.20, 0.15, 0.25]
w_total = 0.10 + 0.20 + 0.15 + 0.25
         = 0.70
```

```
mid = (0 + 4) // 2 = 2
```

```
left half = nodes 0:2 → weights [0.10, 0.20]
```

```
right half = nodes 2:4 → weights [0.15, 0.25]
```

```
w_left = 0.10 + 0.20 = 0.30
f1 = 0.30 / 0.70 ≈ 0.429
```



$$w_i^k = \sum_{j=2^{N-i}(k-1)+1}^{2^{N-i}k} w_N^j$$

$$w_i^k = w_{i+1}^{2k-1} + w_{i+1}^{2k}$$

$$f_{i+1}^{2k-1} = w_{i+1}^{2k-1} / w_i^k.$$

```
def regularization(self, lambda_reg=1e-6):
    """
    Regularization term from Eq. 34
    L(z) = (sum_j ReLU(z_N^j) - \xi)^2
    where \xi = 2^(N-1) from Eq. 48
    """
    z_act = torch.relu(self.z)
    xi = 2**(self.depth - 1)
    return 0.5 * lambda_reg * (torch.sum(z_act) - xi)**2
```

```
print("✓ MaterialNetwork defined (CORRECTED)")
```

✓ MaterialNetwork defined (CORRECTED)

Code Cell 6 : Prepare Data

What this block does

This block preprocesses DNS-generated material data and converts it into a format suitable for training the neural network.

How it works

- Phase stiffness matrices and effective stiffness matrices are extracted from the dataset.
- Matrix inversion is used to convert stiffness tensors into compliance tensors:

$$D = C^{-1}$$

- Symmetric 3×3 compliance matrices are compressed into 6-component Mandel vectors.
- The dataset is split into:
 - training set,
 - validation set,
 - testing set.
- PyTorch tensors and DataLoaders are created for efficient batch-based GPU training.

Code Cell 6 : Prepare Data

```
# =====  
# Cell 6: Prepare Data (CORRECTED)  
# =====
```

```
def prepare_data(data):  
    """  
    Prepare training data from DNS results  
    Since your data is already 3x3 in Mandel notation:  
    [C11, C12, C13; C12, C22, C23; C13, C23, C33]  
  
    Convert to compliance vectors [D11, D12, D13, D22, D23, D33]  
    """  
    n = data['C_eff_data'].shape[2]  
    X1, X2, Y = [], [], []  
  
    print(f"Processing {n} samples...")  
  
    for i in range(n):  
        # Phase 1 stiffness (already 3x3) -> compliance  
        C_p1 = data['C_p1'][:, :, i]  
        D1_mat = np.linalg.inv(C_p1)  
        # Convert to Mandel 6-vector  
        D1 = np.array([  
            D1_mat[0, 0], D1_mat[0, 1], D1_mat[0, 2],  
            D1_mat[1, 1], D1_mat[1, 2], D1_mat[2, 2]  
        ], dtype=np.float32)
```

```
# Phase 2 stiffness (already 3x3) -> compliance
```

```
C_p2 = data['C_p2'][:, :, i]  
D2_mat = np.linalg.inv(C_p2)  
D2 = np.array([  
    D2_mat[0, 0], D2_mat[0, 1], D2_mat[0, 2],  
    D2_mat[1, 1], D2_mat[1, 2], D2_mat[2, 2]  
], dtype=np.float32)
```

```
# Target: effective stiffness (already 3x3) -> compliance
```

```
C_eff = data['C_eff_data'][:, :, i]  
D_target_mat = np.linalg.inv(C_eff)  
D_target = np.array([  
    D_target_mat[0, 0], D_target_mat[0, 1], D_target_mat[0, 2],  
    D_target_mat[1, 1], D_target_mat[1, 2], D_target_mat[2, 2]  
], dtype=np.float32)
```

```
X1.append(D1)  
X2.append(D2)  
Y.append(D_target)
```

```
return np.array(X1), np.array(X2), np.array(Y)
```

```
# Prepare data
```

```
X1, X2, Y = prepare_data(data)  
print(f"\n✓ Prepared {len(X1)} samples")
```

Code Cell 6 : Prepare Data

```
# Split data
n = len(X1)
indices = np.random.permutation(n)
train_idx = indices[:int(0.6 * n)]
val_idx = indices[int(0.6 * n):int(0.8 * n)]
test_idx = indices[int(0.8 * n):]

# Convert to tensors
X1_train = torch.tensor(X1[train_idx], device=device, dtype=torch.float32)
X2_train = torch.tensor(X2[train_idx], device=device, dtype=torch.float32)
Y_train = torch.tensor(Y[train_idx], device=device, dtype=torch.float32)

X1_val = torch.tensor(X1[val_idx], device=device, dtype=torch.float32)
X2_val = torch.tensor(X2[val_idx], device=device, dtype=torch.float32)
Y_val = torch.tensor(Y[val_idx], device=device, dtype=torch.float32)

X1_test = torch.tensor(X1[test_idx], device=device, dtype=torch.float32)
X2_test = torch.tensor(X2[test_idx], device=device, dtype=torch.float32)
Y_test = torch.tensor(Y[test_idx], device=device, dtype=torch.float32)

# Create data loaders
batch_size = 64
train_dataset = TensorDataset(X1_train, X2_train, Y_train)
val_dataset = TensorDataset(X1_val, X2_val, Y_val)
test_dataset = TensorDataset(X1_test, X2_test, Y_test)

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
```

```
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)

print(f"Train: {len(train_dataset)}, Val: {len(val_dataset)}, Test: {len(test_dataset)}")

# Verify data quality
print("\nVerifying data quality...")
print(f"Phase 1 D11 range: [{X1[:, 0].min():.4f}, {X1[:, 0].max():.4f}]")
print(f"Phase 2 D11 range: [{X2[:, 0].min():.4f}, {X2[:, 0].max():.4f}]")
print(f"Target D11 range: [{Y[:, 0].min():.4f}, {Y[:, 0].max():.4f}]")
```

Processing 500 samples...

✓ Prepared 500 samples
Train: 300, Val: 100, Test: 100

Verifying data quality...
Phase 1 D11 range: [0.0743, 14.5397]
Phase 2 D11 range: [0.0002, 3471.8699]
Target D11 range: [0.0555, 21.2488]

Code Cell 7 : Trainer

What this block does

This block defines the complete training framework for optimizing the Deep Material Network.

How it works

- The Adam optimizer updates trainable parameters using gradient descent.
- The loss function is based on the normalized squared Frobenius norm:

$$\frac{\|D^{dns} - D^{mn}\|_F^2}{\|D^{dns}\|_F^2}$$

- Regularization is added to improve stability and prevent overfitting.
- Training and validation loops are implemented separately.
- Gradient clipping is used to stabilize optimization.
- A learning-rate scheduler automatically reduces the learning rate when validation loss plateaus.
- Loss histories are stored and visualized using plots.

Code Cell 7 : Trainer

```
# =====  
# Cell 7: Trainer (CORRECTED for newer PyTorch)  
# =====  
  
class Trainer:  
    def __init__(self, model, lr=0.001, lambda_reg=1e-6):  
        self.model = model  
        self.lambda_reg = lambda_reg  
        self.optimizer = optim.Adam(model.parameters(), lr=lr)  
        # Fixed: removed 'verbose' parameter for newer PyTorch versions  
        self.scheduler = optim.lr_scheduler.ReduceLROnPlateau(  
            self.optimizer, mode='min', factor=0.5, patience=50  
        )  
        self.train_losses = []  
        self.val_losses = []  
  
    def compute_loss(self, pred, target):  
        """  
        Eq. 30: Normalized squared Frobenius norm  
         $C(z, \theta) = (1/2) * ||D^{dns} - D^{mn}||^2 / ||D^{dns}||^2$   
        """  
        batch_size = pred.shape[0]  
  
        # Compute difference  
        diff = pred - target  
  
        # Frobenius norm squared of error  
        frob_error = torch.sum(diff**2, dim=1) # [batch]
```

```
# Frobenius norm squared of target (for normalization)  
frob_target = torch.sum(target**2, dim=1) + 1e-10 # [batch]  
  
# Normalized loss per sample  
loss_per_sample = frob_error / frob_target  
  
return loss_per_sample.mean()  
  
def total_loss(self, pred, target):  
    """  
    Combined loss: data term + regularization (Eq. 33)  
    """  
    data_loss = self.compute_loss(pred, target)  
    reg_loss = self.model.regularization(self.lambda_reg)  
    return data_loss + reg_loss  
  
def train_epoch(self, loader):  
    self.model.train()  
    total_loss = 0  
  
    for X1, X2, Y in loader:  
        self.optimizer.zero_grad()  
  
        # Forward pass  
        pred = self.model(X1, X2)  
        loss = self.total_loss(pred, Y)
```


Code Cell 7 : Trainer

```
# Backward pass
loss.backward()
torch.nn.utils.clip_grad_norm_(self.model.parameters(), 1.0)
self.optimizer.step()

total_loss += loss.item()

return total_loss / len(loader)

def validate(self, loader):
    self.model.eval()
    total_loss = 0

    with torch.no_grad():
        for X1, X2, Y in loader:
            pred = self.model(X1, X2)
            loss = self.compute_loss(pred, Y)
            total_loss += loss.item()

    return total_loss / len(loader)

def train(self, train_loader, val_loader, epochs=500):
    print(f"\n{'='*60}")
    print(f"TRAINING ON {device}")
    print(f"{'='*60}")

    start_time = time.time()
```

```
for epoch in range(epochs):
    train_loss = self.train_epoch(train_loader)
    val_loss = self.validate(val_loader)

    self.train_losses.append(train_loss)
    self.val_losses.append(val_loss)

    # Step the scheduler with validation loss
    self.scheduler.step(val_loss)

    if (epoch + 1) % 50 == 0:
        elapsed = time.time() - start_time
        current_lr = self.optimizer.param_groups[0]['lr']
        print(f"Epoch {epoch+1:3d}: LR={current_lr:.6f}, Train={train_loss:.6f}, "
              f"Val={val_loss:.6f}, Time={elapsed:.0f}s")

    total_time = time.time() - start_time
    print(f"\n✓ Training complete in {total_time/60:.2f} minutes")

def plot(self):
    plt.figure(figsize=(10, 5))
    plt.plot(self.train_losses, label='Train Loss')
    plt.plot(self.val_losses, label='Val Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.title('Training History')
    plt.legend()
    plt.yscale('log')
    plt.grid(True, alpha=0.3)
    plt.show()

print("✓ Trainer defined (FIXED for newer PyTorch)")
```

✓ Trainer defined (FIXED for newer PyTorch)

Code Cell 8 : Network Training

What this block does

This block initializes the Deep Material Network and starts the training process.

How it works

- GPU memory cache is cleared before training.
- A MaterialNetwork model is created with a chosen tree depth.
- Initial phase compliances are provided as reference tensors.
- The Trainer object is initialized with:
 - learning rate,
 - regularization strength.
- The model is trained over multiple epochs using training and validation datasets.
- Training and validation losses are plotted to monitor convergence and model performance.

Code Cell 8 : Network Training

```
# =====  
# Cell 8: Train the Network  
# =====  
  
# Clear GPU cache  
if torch.cuda.is_available():  
    torch.cuda.empty_cache()  
  
# Create network  
depth = 5 # Start with depth 3-5  
model = MaterialNetwork(  
    depth=depth,  
    D1_init=X1_train[0], # Reference compliance for phase 1  
    D2_init=X2_train[0]  # Reference compliance for phase 2  
) .to(device)  
  
print(f"Model depth: {depth}")  
print(f"Bottom nodes: {model.n_bottom}")  
print(f"Trainable parameters: {sum(p.numel() for p in model.parameters() if p.requires_grad)}")  
  
# Create trainer  
trainer = Trainer(model, lr=0.001, lambda_reg=1e-6)  
  
# Train  
trainer.train(train_loader, val_loader, epochs=3000)  
trainer.plot()
```

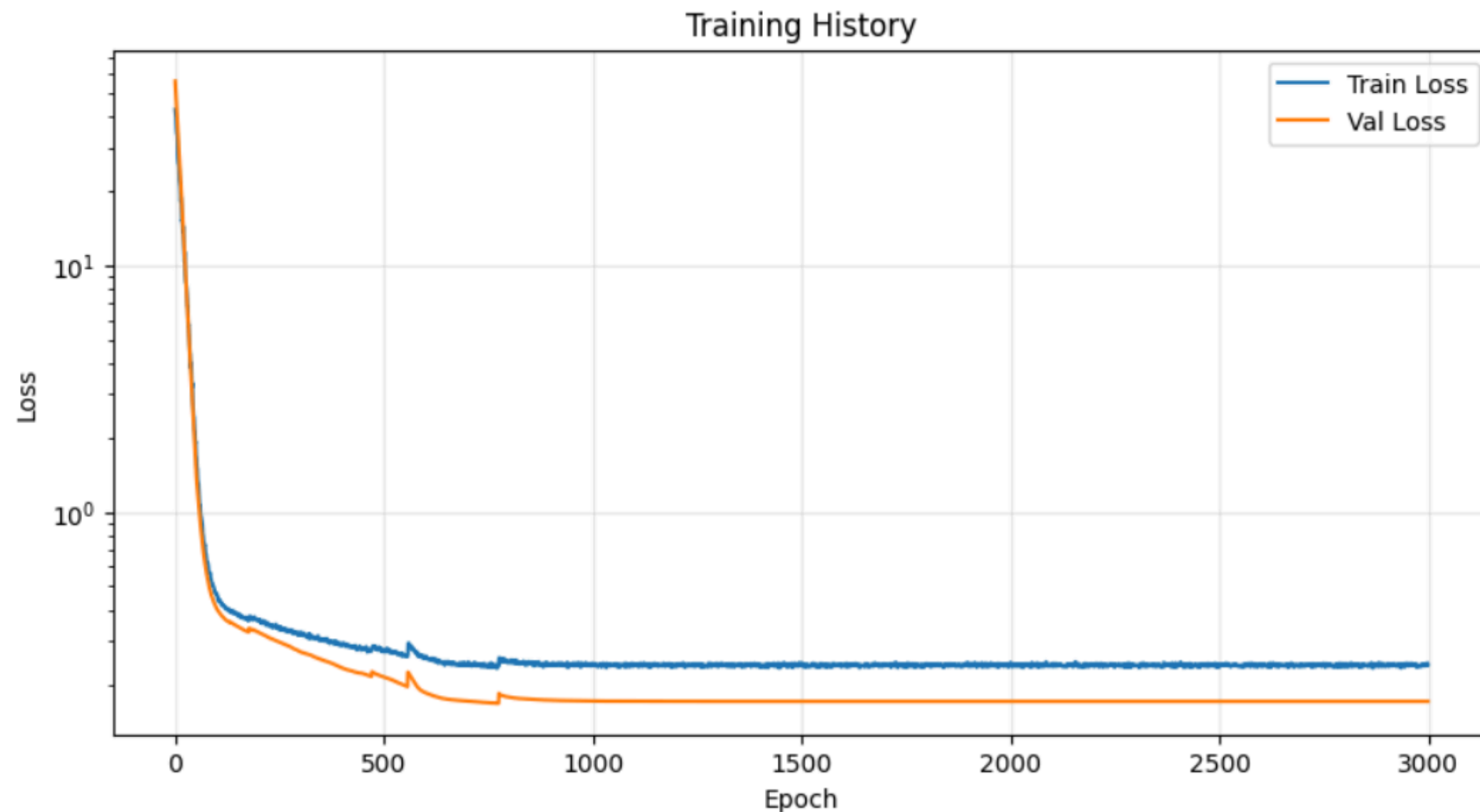
Model depth: 5
Bottom nodes: 32
Trainable parameters: 95

```
=====
TRAINING ON cuda
=====
Epoch  50: LR=0.001000, Train=1.932677, Val=1.578989, Time=86s
Epoch 100: LR=0.001000, Train=0.465565, Val=0.406734, Time=168s
Epoch 150: LR=0.001000, Train=0.383826, Val=0.343487, Time=250s
Epoch 200: LR=0.001000, Train=0.363633, Val=0.325261, Time=330s
Epoch 250: LR=0.001000, Train=0.342939, Val=0.297042, Time=411s
Epoch 300: LR=0.001000, Train=0.324743, Val=0.270945, Time=491s
Epoch 350: LR=0.001000, Train=0.300227, Val=0.251712, Time=572s
Epoch 400: LR=0.001000, Train=0.290192, Val=0.233187, Time=653s
Epoch 450: LR=0.001000, Train=0.278300, Val=0.221141, Time=733s
Epoch 500: LR=0.001000, Train=0.273929, Val=0.214507, Time=812s
Epoch 550: LR=0.001000, Train=0.262938, Val=0.198009, Time=893s
Epoch 600: LR=0.001000, Train=0.254737, Val=0.184857, Time=972s
Epoch 650: LR=0.001000, Train=0.242129, Val=0.174224, Time=1052s
Epoch 700: LR=0.001000, Train=0.242497, Val=0.171006, Time=1132s
Epoch 750: LR=0.001000, Train=0.234700, Val=0.168376, Time=1214s
Epoch 800: LR=0.001000, Train=0.246073, Val=0.177363, Time=1293s
Epoch 850: LR=0.000500, Train=0.242208, Val=0.174058, Time=1375s
Epoch 900: LR=0.000250, Train=0.242184, Val=0.172329, Time=1454s
Epoch 950: LR=0.000125, Train=0.242478, Val=0.171545, Time=1534s
Epoch 1000: LR=0.000063, Train=0.240948, Val=0.171022, Time=1615s
Epoch 1050: LR=0.000031, Train=0.237900, Val=0.170744, Time=1695s
Epoch 1100: LR=0.000016, Train=0.239694, Val=0.170603, Time=1774s
Epoch 1150: LR=0.000008, Train=0.238273, Val=0.170532, Time=1855s
```


Code Cell 8 : Network Training

```
Epoch 1200: LR=0.000004, Train=0.235441, Val=0.170496, Time=1935s
Epoch 1250: LR=0.000002, Train=0.238104, Val=0.170477, Time=2016s
Epoch 1300: LR=0.000001, Train=0.239914, Val=0.170468, Time=2097s
Epoch 1350: LR=0.000000, Train=0.241756, Val=0.170464, Time=2183s
Epoch 1400: LR=0.000000, Train=0.240744, Val=0.170462, Time=2264s
Epoch 1450: LR=0.000000, Train=0.241433, Val=0.170460, Time=2345s
Epoch 1500: LR=0.000000, Train=0.238557, Val=0.170460, Time=2428s
Epoch 1550: LR=0.000000, Train=0.241992, Val=0.170460, Time=2509s
Epoch 1600: LR=0.000000, Train=0.239243, Val=0.170460, Time=2591s
Epoch 1650: LR=0.000000, Train=0.238713, Val=0.170460, Time=2670s
Epoch 1700: LR=0.000000, Train=0.238642, Val=0.170460, Time=2751s
Epoch 1750: LR=0.000000, Train=0.234561, Val=0.170460, Time=2831s
Epoch 1800: LR=0.000000, Train=0.242622, Val=0.170460, Time=2911s
Epoch 1850: LR=0.000000, Train=0.240750, Val=0.170460, Time=2991s
Epoch 1900: LR=0.000000, Train=0.241148, Val=0.170460, Time=3072s
Epoch 1950: LR=0.000000, Train=0.240695, Val=0.170460, Time=3151s
Epoch 2000: LR=0.000000, Train=0.238208, Val=0.170460, Time=3231s
Epoch 2050: LR=0.000000, Train=0.234709, Val=0.170460, Time=3311s
Epoch 2100: LR=0.000000, Train=0.242372, Val=0.170460, Time=3390s
Epoch 2150: LR=0.000000, Train=0.235863, Val=0.170460, Time=3469s
Epoch 2200: LR=0.000000, Train=0.236550, Val=0.170460, Time=3552s
Epoch 2250: LR=0.000000, Train=0.239000, Val=0.170460, Time=3634s
Epoch 2300: LR=0.000000, Train=0.239297, Val=0.170460, Time=3715s
Epoch 2350: LR=0.000000, Train=0.235680, Val=0.170460, Time=3795s
Epoch 2400: LR=0.000000, Train=0.237108, Val=0.170460, Time=3876s
Epoch 2450: LR=0.000000, Train=0.238518, Val=0.170460, Time=3957s
Epoch 2500: LR=0.000000, Train=0.238363, Val=0.170460, Time=4039s
Epoch 2550: LR=0.000000, Train=0.236375, Val=0.170460, Time=4122s
Epoch 2600: LR=0.000000, Train=0.238710, Val=0.170460, Time=4203s
Epoch 2650: LR=0.000000, Train=0.239865, Val=0.170460, Time=4287s
Epoch 2700: LR=0.000000, Train=0.238858, Val=0.170460, Time=4370s
Epoch 2750: LR=0.000000, Train=0.243005, Val=0.170460, Time=4454s
Epoch 2800: LR=0.000000, Train=0.238639, Val=0.170460, Time=4537s
Epoch 2850: LR=0.000000, Train=0.239721, Val=0.170460, Time=4619s
Epoch 2900: LR=0.000000, Train=0.240332, Val=0.170460, Time=4700s
Epoch 2950: LR=0.000000, Train=0.239788, Val=0.170460, Time=4782s
Epoch 3000: LR=0.000000, Train=0.240134, Val=0.170460, Time=4863s
```

✓ Training complete in 81.04 minutes



Code Cell 9 : Test on Unseen Data

What this block does

This block evaluates the trained model on unseen test samples and measures prediction accuracy.

How it works

- The model is switched to evaluation mode.
- Gradient computation is disabled to improve efficiency.
- Predictions are generated for test samples.
- Error is computed using normalized Frobenius norm loss.
- Average test loss is calculated to evaluate generalization performance.
- Sample predictions are printed alongside target values for qualitative comparison between predicted and true compliance tensors.

Code Cell 10 : Saving the Trained Model

This block saves the trained PyTorch model weights and automatically downloads the file to the local system for future use or deployment.

Code Cell 9 : Test on Unseen Data

```
# =====
# Cell 9: Test on Unseen Data
# =====

print("=" * 60)
print("TESTING ON UNSEEN DATA")
print("=" * 60)

model.eval()
test_losses = []

with torch.no_grad():
    for X1, X2, Y in test_loader:
        pred = model(X1, X2)
        for i in range(len(Y)):
            diff = pred[i] - Y[i]
            frob_error = torch.sum(diff**2)
            frob_target = torch.sum(Y[i]**2) + 1e-10
            loss = frob_error / frob_target
            test_losses.append(loss.item())

print(f"\nAverage Test Loss: {np.mean(test_losses):.6f} ({np.mean(test_losses)*100:.2f}%)")

print("\n" + "=" * 60)
print("SAMPLE PREDICTIONS")
print("=" * 60)
```

```
with torch.no_grad():
    for i in range(min(5, len(test_dataset))):
        X1, X2, Y = test_dataset[i]
        pred = model(X1.unsqueeze(0), X2.unsqueeze(0)).squeeze(0)

        diff = pred - Y
        frob_error = torch.sum(diff**2)
        frob_target = torch.sum(Y**2) + 1e-10
        loss = frob_error / frob_target

        print(f"\nSample {i+1}:")
        print(f"  Target:    [{Y[0]:7.4f}, {Y[1]:7.4f}, {Y[2]:7.4f}, {Y[3]:7.4f}, {Y[4]:7.4f}, {Y[5]:7.4f}]")
        print(f"  Predicted: [{pred[0]:7.4f}, {pred[1]:7.4f}, {pred[2]:7.4f}, {pred[3]:7.4f}, {pred[4]:7.4f}, {pred[5]:7.4f}]")
        print(f"  Error: {loss.item():.6f} ({loss.item()*100:.2f}%)")
```

TESTING ON UNSEEN DATA	SAMPLE PREDICTIONS
Average Test Loss: 0.226615 (22.66%)	Sample 1: Target: [2.3414, -0.0347, -0.0029, 0.2342, 0.0001, 0.9506] Predicted: [0.6642, -0.1537, 0.0547, 0.3014, -0.0583, 0.4895] Error: 0.473628 (47.36%)
	Sample 2: Target: [6.4611, -2.4754, 0.0950, 9.7385, -0.8662, 7.8388] Predicted: [6.7729, -3.2559, 0.2870, 6.2419, 0.1171, 4.5563] Error: 0.120592 (12.06%)
	Sample 3: Target: [5.8820, -0.6140, -0.0002, 0.6923, 0.0010, 2.2929] Predicted: [2.8641, -0.7732, 0.6602, 0.7296, -0.3465, 0.9036] Error: 0.285454 (28.55%)
	Sample 4: Target: [1.0074, -0.5695, -0.0007, 4.4084, -0.0002, 1.5590] Predicted: [1.1471, -0.4082, 0.0120, 2.9486, 0.0125, 0.9871] Error: 0.107911 (10.79%)
	Sample 5: Target: [0.2773, -0.0643, 0.0009, 0.5785, -0.0006, 0.5128] Predicted: [0.1229, -0.0100, -0.0967, 0.6000, -0.0195, 0.4200] Error: 0.067384 (6.74%)

Code Cell 10 : Save Model

```
# =====  
# Cell 10: Save Model  
# =====
```

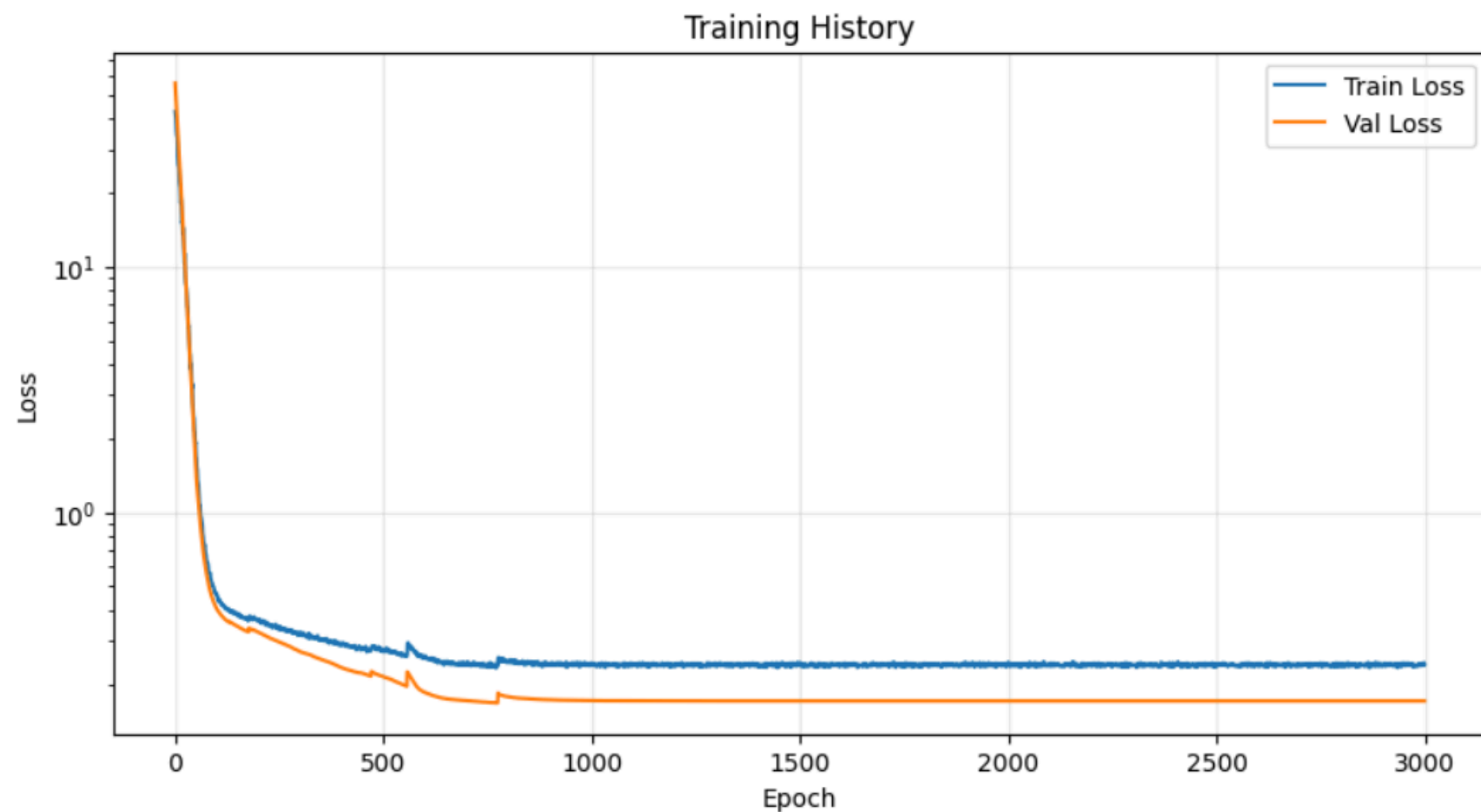
```
torch.save(model.state_dict(), 'dmn_pytorch.pt')  
print("✓ Model saved to dmn_pytorch.pt")
```

```
from google.colab import files  
files.download('dmn_pytorch.pt')  
print("✓ Download initiated")
```

```
✓ Model saved to dmn_pytorch.pt  
✓ Download initiated
```

Conclusion:

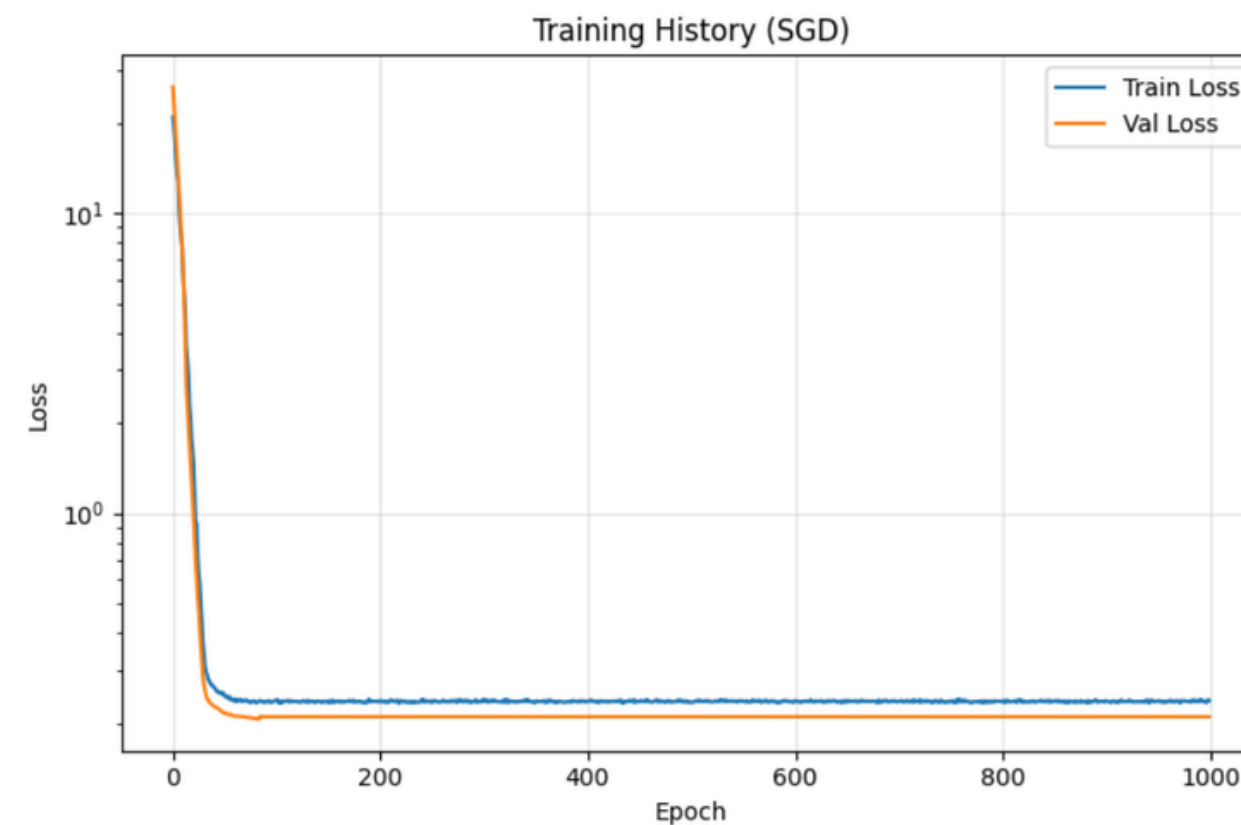
Best Result: PyTorch Implementation using ADAM Optimizer



Training loss converged till ~ 0.24
Validation loss converged till ~ 0.17
after 1000 epochs.

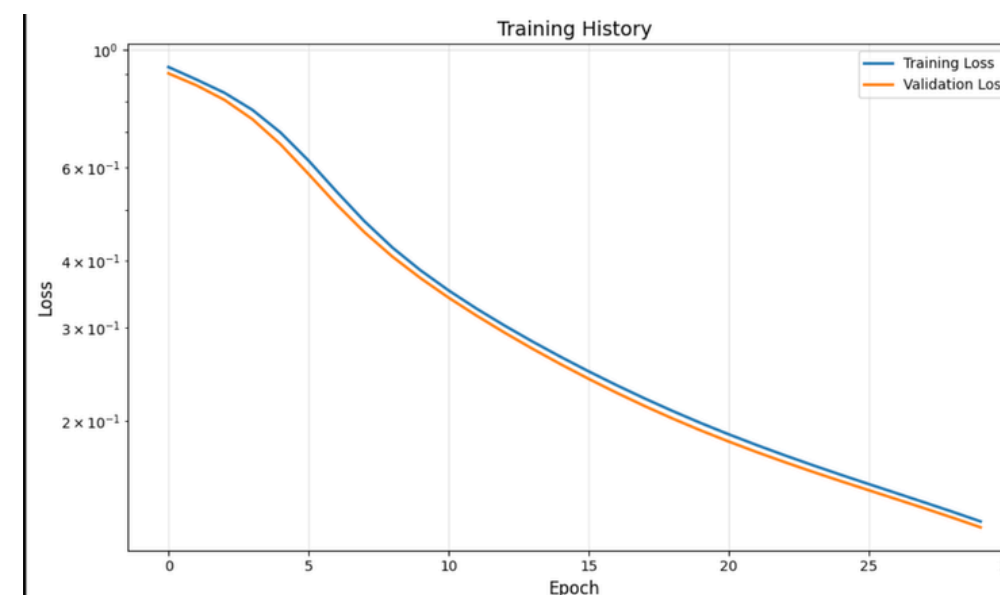
Other Attempts:

PyTorch Implementation using SGD Optimizer



The model converged around epoch 100, after which both training and validation loss remained nearly constant because the learning rate became almost zero, so the model parameters stopped updating significantly.

Scratch Implementation (Using Finite Differences for Gradient Calculation)



- Needs 50-60 hours for training on big datasets and till 1000 epochs.
- Ran it till 30 epochs for smaller depth .